

Inteligencia Artificial

Práctica 2

Luis Airam Saavedra Paiseo

Abril 2026

Índice

Contenido

Índice	1
1. Introducción	2
1.1 Descripción del Problema	2
2. Diseño de Algoritmos	3
2.1 Algoritmo de Búsqueda en Anchura (BFS)	3
2.2 Algoritmo de Búsqueda en Profundidad (DFS)	4
2.3 Algoritmo A* Mejorado	5
2.4 Trazas de ejecución	6
3. Análisis de Resultados	8
3.1 Análisis de Optimalidad	8
3.2 Análisis de A*	8
4. Conclusiones	9
5. Anexo	9

1. Introducción

Esta práctica está basada en algo similar a la primera práctica, partimos de un laberinto y el Agente tiene las mismas restricciones de movimiento que en la primera, es decir, no puede retroceder por su estela ni atravesar muros, siguiendo el comportamiento tipo tron.

1.1 Descripción del Problema

El proyecto consiste en 4 mapas de caracteres, uno de tamaño pequeño con una "trampa" en forma de callejón, otro más grande con más posibilidades de rutas que lleven a la salida, un tercer mapa que es extraído de la ayuda de la propia práctica y un último mapa diseñado especialmente para el comportamiento de 2 algoritmos concretos. El objetivo del Agente es conseguir llegar desde el punto de inicio A hasta la salida S, evitando las paredes # y las estelas que irán dejando allá por donde pasen, encontrando siempre que pueda y sea posible el camino óptimo.

```
static final String mapaText01 =  
    "#####\n" +  
    "#A      #\n" +  
    "# ##### #\n" +  
    "# #   # #\n" +  
    "#      S #\n" +  
    "#####";
```

Mapa 1: Mapa pequeño con callejón trampa al Este desde la posición inicial.

```
static final String mapaText02 =  
    "#####\n" +  
    "#A  #           #\n" +  
    "# # # ##### #\n" +  
    "# #   #   #   #\n" +  
    "# ##### ### #\n" +  
    "#      #   #   #\n" +  
    "# ### ##### # #\n" +  
    "#   #           #\n" +  
    "# ##### S #\n" +  
    "#####";
```

Mapa 2: Mapa grande con múltiples rutas y mayor complejidad.

```
static final String mapaText03 =  
    "#####\n" +  
    "#E      #\n" +  
    "# ### #\n" +  
    "#   #S#\n" +  
    "#####";
```

Mapa 3: Mapa pequeño con 1 único posible camino

```
static final String mapaText04 =  
    "#####\n" +
```

"#S #\n" +
"# #\n" +
"# A#\n" +
"#####";

Mapa 4: Mapa abierto con muchos caminos posibles

El proyecto base gira en torno al método esTransitable que es igual al de la práctica 1, en esta práctica tenemos la ventaja de que el Agente “piensa” antes de mover, por lo que el único método trivial que necesitamos será este, el resto de los métodos como el “dibujar” o “moverAgente” serán de ayuda para la ejecución únicamente. El único que ayuda en la lógica será este.

```
public boolean esTransitable(int f, int c) {
    if (f < 0 || f >= filas || c < 0 || c >= cols) return false;
    return grid[f][c] != '#' && grid[f][c] != '.';
}
```

Código 1: Método que permite saber si una casilla con coordenada fila f y columna c es transitable o no.

2. Diseño de Algoritmos

Para el diseño de los distintos algoritmos se han utilizado diversas estructuras de datos tales como Listas enlazadas (LinkedLists), Pilas (Stacks) o Colas de Prioridad (PriorityQueue) de forma que según el algoritmo que estemos diseñando, utilizaremos una u otra para manejar el comportamiento de los nodos visitados entre otros.

2.1 Algoritmo de Búsqueda en Anchura (BFS)

El primer Algoritmo implementado ha sido el BFS, característico porque “barre” un nivel completo antes de pasar al siguiente, esto es, en un árbol de nodos, hasta que no explora todos los nodos del nivel actual, no pasará a analizar el siguiente nivel. De esta forma se garantiza que encuentra la solución óptima siempre que los costes sean uniformes, más adelante se entrará en detalle sobre esta característica. Este algoritmo utiliza para todas sus estructuras Listas enlazadas puesto que el orden es ir sacando el primero que entra, comportamiento típico FIFO, se usan las enlazadas por ventajas de la estructura frente a la Lista simple y por comodidad comparado a ArrayList.

Este algoritmo ha sido implementado a partir del pseudocódigo del temario de la asignatura, quedando de manera resumida en el siguiente código:

```
Public List<String> resolver(Entorno mapa)
List<Nodo>Abiertos=new LinkedList<>(); //Estructura abiertos

List<Nodo>Cerrados=new LinkedList<>(); //Estructura cerrados

Nodo inicial=new Nodo(f,c,null,null,0,0); //nodo inicial

Abiertos.add(inicial) //añadimos primer nodo a abiertos
While (abiertos.size() > 0){
    Nodo actual=abiertos.removeFirst(); //guardamos el actual para comparaciones
    Cerrados.add(actual); //se ha expandido el nodo, pasa de abiertos a cerrados
    If (mapa.esMeta(actual.f,actual.c)
    Return reconstruirCamino(actual,"BFS"); //si es meta, reconstruimos camino
    Else{
        Sucesores=getSucesores(Actual,mapa); //si no es meta, obtenemos sucesores
        Sucesores=quitarRepetidos(sucesores,cerrados,abiertos); //tratamiento repetidos
        Abiertos.addAll(sucesores);} //añadimos todos los sucesores y repetimos proceso
    } //fin de while
Return null; //si no encuentra salida devuelve nulo
}
```

Código 1: Algoritmo BFS base resumido.

2.2 Algoritmo de Búsqueda en Profundidad (DFS)

Este Algoritmo se comporta a groso modo exactamente igual que el BFS, pero este en vez de utilizar Listas enlazadas hará uso de Pilas debido a que tiene un comportamiento LIFO. El comportamiento del DFS es profundizar una rama del árbol hasta no poder continuar, es decir, no encontrar sucesores. Garantiza una solución más rápida que el BFS, pero no una más óptima en muchos de los casos, pues deja sin evaluar muchas ramas del árbol que pueden contener una solución mejor que la elegida por el algoritmo.

```
Public List<String> resolver(Entorno mapa)

Stack<Nodo>Abiertos=new Stack<>(); //Estructura abiertos

Stack<Nodo>Cerrados=new Stack<>(); //Estructura cerrados

Nodo inicial=new Nodo(f,c,null,null,0,0); //nodo inicial

Abiertos.add(inicial) //añadimos primer nodo a abiertos

While (abiertos.size() > 0){

Nodo actual=abiertos.pop(); //guardamos el actual para comparaciones

Cerrados.add(actual); //se ha expandido el nodo, pasa de abiertos a cerrados

If (mapa.esMeta(actual.f,actual.c)

Return reconstruirCamino(actual,"DFS"); //si es meta, reconstruimos camino

Else{

Sucesores=getSucesores(Actual,mapa); //si no es meta, obtenemos sucesores

Sucesores=quitarRepetidos(sucesores,cerrados,abiertos); //tratamiento repetidos

Int n= sucesores.size(); //tamaño de sucesores para insertar en abiertos

For (int i=0; i<n;i++){

Abiertos.push(sucesores.pop());} //añadimos todos los sucesores y repetimos proceso

} //fin de while

Return null; //si no encuentra salida devuelve nulo

}
```

Código 2: Algoritmo DFS base resumido

2.3 Algoritmo A*

Este Algoritmo es el más sofisticado de todos los que se han implementado hasta ahora, pues tendrá a su disposición el uso de heurísticas, o estrategias de ayuda para la toma de decisiones que en base a un cálculo de la variable “h” nos ayudará a saber cuánto de óptimo es continuar por un camino o por otro. Este Algoritmo puede implementar tantas estrategias como heurísticas se le puedan añadir, y se rige por la fórmula de:

$$F(n)=g(n)+h(n)$$

Siendo g(n) el coste real de llegar hasta ese nodo por el camino actual y h(n) el coste aproximado de llegar a la meta desde ese mismo nodo según la heurística elegida, en nuestro caso hemos utilizado la Distancia Manhattan la cual calcula la distancia en línea recta desde el nodo actual hasta la meta y utiliza la siguiente fórmula para ello:

$$|x_1-x_2| + |y_1-y_2|$$

De esta forma, este Algoritmo se rige por elegir aquellos nodos de menor f(n), y en caso de coincidir en valor nos guiaremos por aquellos con un menor valor de h(n) que son aquellos más “óptimos”, haciendo que este algoritmo garantice una solución óptima sin condiciones necesarias como el BFS o DFS.

Para la implementación de este algoritmo también nos hemos basado en el pseudocódigo del temario de la asignatura, quedando de la siguiente manera:

```
Public List<String> resolver(Entorno mapa)

Queue<Nodo>Abiertos=new PriorityQueue<>(); //Estructura abiertos

Queue<Nodo>Cerrados=new PriorityQueue<>(); //Estructura cerrados

Nodo inicial=new Nodo(f,c,null,null,0,0); //nodo inicial

Abiertos.add(inicial) //añadimos primer nodo a abiertos

While (abiertos.size() > 0){

Nodo actual=abiertos.remove(); //guardamos el actual para comparaciones

Cerrados.add(actual); //se ha expandido el nodo, pasa de abiertos a cerrados
```

```

If (mapa.esMeta(actual.f,actual.c)
Return reconstruirCamino(actual,"A*"); //si es meta, reconstruimos camino
Else{
Sucesores=getSucesores(Actual,mapa); //si no es meta, obtenemos sucesores
Sucesores=quitarRepetidos(sucesores,cerrados,abiertos); //tratamiento repetidos
Abiertos.addAll(sucesores);} //añadimos todos los sucesores y repetimos proceso
} //fin de while
Return null; //si no encuentra salida devuelve nulo
}

```

Código 3: Algoritmo A* base resumido.

2.3 Algoritmo A* Mejorado

Este Algoritmo, el cual se ha optado por llamar A* ponderado intenta compensar los costes asimétricos de los movimientos siguiendo la siguiente regla:

Calculamos la distancia para movimientos de filas y columnas ($f = \text{mapa.meta} - f$ y lo mismo para c), si f es positiva indica que tenemos que ir hacia el sur para llegar a la meta, de igual manera si c es positiva indica que debemos ir hacia la derecha, pero a la que el Agente deba ir hacia el Norte o hacia el Oeste, se encontrará con que se han penalizado estos movimientos, haciendo que la h no cueste lo que cuesta realmente, sino que el movimiento N y O costarán el doble, haciendo que nuestra $h(n)$ no subestime ni sobreestime $h'(n)$, haciéndola optable de utilizar. Además de que mejora el A* original, puesto que este tiene una heurística menos óptima en cuanto a si sobreestima o subestima $h'(n)$.

El código de este Algoritmo es el mismo que el del A* cambiando únicamente el cálculo de la h , que sigue la siguiente fórmula:

```

((Despf>=0) ? despF : Math.abs(despF*2))
+ ((despC >=0) ? despC : Math.abs(despC*2))

```

Siendo despF y despC los desplazamientos mencionados anteriormente.

Todos los algoritmos a su vez hacen uso de 2 métodos, “getSucesores” y “quitarRepetidos”, todos obtienen los sucesores de la misma manera pero el A* añade el cálculo de la $h(n)$, este método se vería de la siguiente manera para ambos casos:

```

Private Queue<Nodo> getSucesores(Nodo Actual, Entorno mapa){

Queue<Nodo> sucesores=new PriorityQueue<>();

Int f=Actual.f;

Int c=Actual.c;

Int g=Actual.g; //valores f c y g del nodo en un momento concreto.

If (mapa.estransitable(f-1,c){

Int h=calcularH(f-1,c,mapa); //función auxiliar para calcular el valor de h

Sucesores.add(new Nodo(f-1,Actual,"N",g+super.costeIZQ_ARRIBA,h));

//IDEM para las otras 3 direcciones cambiando suer.costeIZQ_ARRIBA por super.costeDER_ABAJO

//en el caso de que la dirección sea S o E

```

Código 4: Método auxiliar de obtener sucesores

Para BFS y DFS, como no tenemos h calculable, ignoraríamos la línea del cálculo de la h y al añadir el nodo pondríamos ‘0’ en ese campo simplemente, manteniendo todo el método igual pero cambiando la estructura utilizada, Stack para DFS y List para BFS.

El método “quitarRepetidos” tiene similitudes entre el BFS y DFS por no decir que repite comportamiento prácticamente idéntico en ambos algoritmos debido a que se comportarán igual pero solo cambiará la estructura utilizada. En el caso del A*, a grandes rasgos el método tiene el mismo código pero internamente se utiliza el método “compareTo” de la clase Nodo con el que nos es posible ordenar por menor f y por menor h en caso de empate del primer valor, haciendo que el método sea prácticamente el mismo pero usando una Queue en vez de una List o Stack.

```

Private Estructura utilizada<String> resolver(Entorno mapa)

Estructura utilizada<Nodo>noRepetidos=new estructura<>();//Estructura de no repetidos

List<Nodo>Cerrados=new LinkedList<>(); //Estructura repetidos igual para todos

For(int i=0; i<sucesores.size();i++){

Nodo s=sucesores.get(i);

If(!Cerrados.contains(s) && !Abiertos.contains(s)){ //Abiertos solo para BFS/DFS

noRepetidos.add/push(s); }//depende de si es cola/lista o pila usamos add o push

Else{

Repetidos.add(s);} //si es un nodo repetido lo añadimos para mostrarlo luego.

If(!repetidos.isEmpty()){ //si tenemos repetidos, los mostramos

System.out.println("Sucesores repetidos descartados : (" + repetidos + ")");

Return noRepetidos;

```

Código 5: Método auxiliar para tratamiento de repetidos.

2.4 Trazas de ejecución

Para mostrar el comportamiento de los 4 agentes voy a hacer uso del mapa “mapaTexto4” que es el que he diseñado para un comportamiento específico de la práctica, de forma que se pueda demostrar el distinto comportamiento de los diferentes algoritmos.

BFS:

```

--- Inicio de la Simulación ---
--- Ciclo 0 ---
#####
#S    #
#     #
#    A#
#####
Iniciando busqueda BFS...
Origen: (3,5)  -> Meta: (1,1)

[PASO 1] Seleccionado: (3,5) [f=0, g=0, h=0]
Generados hijos: [(2,5), (3,4)]
ABIERTOS: [(2,5), (3,4)] | CERRADOS: [(3,5)]

[PASO 2] Seleccionado: (2,5) [f=2, g=2, h=0]
Repetidos descartados: [(3,5)]
Generados hijos: [(1,5), (2,4)]
ABIERTOS: [(3,4), (1,5), (2,4)] | CERRADOS: [(3,5), (2,5)]

[PASO 3] Seleccionado: (3,4) [f=2, g=2, h=0]
Repetidos descartados: [(2,4), (3,5)]
Generados hijos: [(3,3)]
ABIERTOS: [(1,5), (2,4), (3,3)] | CERRADOS: [(3,5), (2,5), (3,4)]

[PASO 4] Seleccionado: (1,5) [f=4, g=4, h=0]
Repetidos descartados: [(2,5)]
Generados hijos: [(1,4)]
ABIERTOS: [(2,4), (3,3), (1,4)] | CERRADOS: [..., (1,5)]

[PASO 5] Seleccionado: (2,4) [f=4, g=4, h=0]
Repetidos descartados: [(1,4), (3,4), (2,5)]
Generados hijos: [(2,3)]
ABIERTOS: [(3,3), (1,4), (2,3)] | CERRADOS: [..., (2,4)]

```

DFS:

```

--- Inicio de la Simulación ---
--- Ciclo 0 ---
#####
#S    #
#     #
#    A#
#####
Iniciando busqueda DFS...
Origen: (3,5)  -> Meta: (1,1)

```

```
[PASO 1] Seleccionado: (3,5) [f=0, g=0, h=0]
Generados hijos: [(2,5), (3,4)]
ABIERTOS: [(3,4), (2,5)] | CERRADOS: [(3,5)]

[PASO 2] Seleccionado: (2,5) [f=2, g=2, h=0]
Repetidos descartados: [(3,5)]
Generados hijos: [(1,5), (2,4)]
ABIERTOS: [(3,4), (2,4), (1,5)] | CERRADOS: [(3,5), (2,5)]

[PASO 3] Seleccionado: (1,5) [f=4, g=4, h=0]
Repetidos descartados: [(2,5)]
Generados hijos: [(1,4)]
ABIERTOS: [(3,4), (2,4), (1,4)] | CERRADOS: [..., (1,5)]

[PASO 4] Seleccionado: (1,4) [f=6, g=6, h=0]
Repetidos descartados: [(2,4), (1,5)]
Generados hijos: [(1,3)]
ABIERTOS: [(3,4), (2,4), (1,3)] | CERRADOS: [..., (1,4)]

[PASO 5] Seleccionado: (1,3) [f=8, g=8, h=0]
Repetidos descartados: [(1,4)]
Generados hijos: [(2,3), (1,2)]
ABIERTOS: [(3,4), (2,4), (1,2), (2,3)] | CERRADOS: [..., (1,3)]
```

A*

--- Inicio de la Simulación ---

--- Ciclo 0 ---

#####

#S #

#

A#

#####

Iniciando búsqueda A*...

Origen: (3,5) -> Meta: (1,1)

```
[PASO 1] Seleccionado: (3,5) [f=12, g=0, h=12]
Generados hijos: [(2,5), (3,4)]
ABIERTOS: [(2,5), (3,4)] | CERRADOS: [(3,5)]

[PASO 2] Seleccionado: (2,5) [f=12, g=2, h=10]
Repetidos descartados: [(3,5)]
Generados hijos: [(1,5), (2,4)]
ABIERTOS: [(1,5), (3,4), (2,4)] | CERRADOS: [..., (2,5)]

[PASO 3] Seleccionado: (1,5) [f=12, g=4, h=8]
Repetidos descartados: [(2,5)]
Generados hijos: [(1,4)]
ABIERTOS: [(1,4), (3,4), (2,4)] | CERRADOS: [..., (1,5)]

[PASO 4] Seleccionado: (1,4) [f=12, g=6, h=6]
Repetidos descartados: [(1,5)]
Generados hijos: [(1,3), (2,4)]
ABIERTOS: [(1,3), (3,4), (2,4), (2,4)] | CERRADOS: [..., (1,4)]

[PASO 5] Seleccionado: (1,3) [f=12, g=8, h=4]
Repetidos descartados: [(1,4)]
Generados hijos: [(1,2), (2,3)]
ABIERTOS: [(1,2), (2,4), (2,4), (3,4), (2,3)] | CERRADOS: [..., (1,3)]
```

A* Ponderado

--- Inicio de la Simulación ---

--- Ciclo 0 ---

#####

#S #

#

A#

#####

Iniciando búsqueda A* Ponderado...

Origen: (3,5) -> Meta: (1,1)

```

[PASO 1] Seleccionado: (3,5) [f=12, g=0, h=12]
Generados hijos: [(2,5), (3,4)]
ABIERTOS: [(2,5), (3,4)] | CERRADOS: [(3,5)]

[PASO 2] Seleccionado: (2,5) [f=12, g=2, h=10]
Repetidos descartados: [(3,5)]
Generados hijos: [(1,5), (2,4)]
ABIERTOS: [(1,5), (3,4), (2,4)] | CERRADOS: [..., (2,5)]

[PASO 3] Seleccionado: (1,5) [f=12, g=4, h=8]
Repetidos descartados: [(2,5)]
Generados hijos: [(1,4)]
ABIERTOS: [(1,4), (3,4), (2,4)] | CERRADOS: [..., (1,5)]

[PASO 4] Seleccionado: (1,4) [f=12, g=6, h=6]
Repetidos descartados: [(1,5)]
Generados hijos: [(1,3), (2,4)]
ABIERTOS: [(1,3), (3,4), (2,4), (2,4)] | CERRADOS: [..., (1,4)]

[PASO 5] Seleccionado: (1,3) [f=12, g=8, h=4]
Repetidos descartados: [(1,4)]
Generados hijos: [(1,2), (2,3)]
ABIERTOS: [(1,2), (2,4), (2,4), (3,4), (2,3)] | CERRADOS: [..., (1,3)]

```

Como podemos observar, en este mapa concreto todos los algoritmos se comportan de manera diferente, interesando sobre todo el crecimiento por capas notorio del BFS frente a la profundización por ramas del DFS, lo cual lleva a que ambos algoritmos encuentren soluciones diferentes en su totalidad. Demostrando así que el BFS lo que hace es crecer por capas a medida que explora y devuelve el mejor camino que encuentre de todos los que encuentre, mientras que el DFS sigue una rama hasta que no puede más, dando ese camino significativamente más extenso que el de BFS, pero expandiendo menos nodos debido a que no garantiza la solución óptima nunca, simplemente garantiza una solución, de existir alguna.

En cuanto a los A*, ambos tienen la misma traza porque el mapa les fuerza a seguir la misma casuística, pero al final de la ejecución el A* Ponderado habrá encontrado el camino en 30 pasos menos aproximadamente que el A* con distancia Manhattan, demostrando así la mejora de eficiencia con esta heurística frente a la proporcionada como heurística genérica.

3. Análisis de Resultados

Para comparar los resultados obtenidos por los 4 algoritmos nos apoyaremos de la siguiente tabla de resultados obtenidos para el mapatexto2, que es el mapa más extenso del que hemos dispuesto.

Algoritmo	Coste Final	Nodos Expandidos	Tamaño Max. ABIERTOS
Anchura (BFS)	32	80	5
Profundidad (DFS)	38	74	5
A* (Manhattan)	32	58	7
A* (Ponderado)	32	62	8

3.1 Análisis de Optimalidad

En este escenario concreto, BFS coincide con A* en el coste final de 32, pero esto es únicamente debido a la estructura particular del mapaTexto2, donde el único camino viable obliga a todos los algoritmos a recorrer las mismas casillas. La pérdida de optimalidad del BFS se hace notoria en mapas con múltiples rutas y costes asimétricos relevantes, debido a que BFS ordena por número de pasos (FIFO) y no por coste acumulado. En un escenario donde existan dos rutas y la de menor número de pasos implique movimientos N u O (coste 2), BFS la elegiría frente a una ruta más larga en pasos, pero compuesta únicamente de movimientos S o E (coste 1), resultando en un camino más caro. A diferencia de A*, BFS no dispone de ningún mecanismo para comparar el coste acumulado entre caminos alternativos, por lo que su garantía de optimalidad solo se mantiene cuando todos los movimientos tienen el mismo coste, condición que no se cumple en esta práctica.

3.2 Análisis de A*

Como hemos podido observar, con la nueva heurística implementada obtenemos en algunos casos el mismo camino que el A* pero en otros ha sido capaz de encontrar una solución más barata en cuanto al coste se refiere que el A* con la distancia Manhattan gracias

al intento de compensación de los costes asimétricos, haciendo que la heurística sea mucho más real y pegada a la realidad del laberinto, impidiendo que el laberinto se deje engañar por falsos caminos “baratos” de coste evitando reabrir nodos innecesariamente, expandiendo lo mínimo indispensable para llegar a la meta. No obstante, sigue siendo un A*, por lo que sigue siendo muy probable que obtengan resultados bastante similares en muchos escenarios con diferentes casuísticas.

4. Conclusiones

En conclusión, hemos podido comprobar que únicamente el A* y el A* Ponderado son capaces de otorgar una solución óptima en todos los casos y escenarios diferentes, siguiéndoles por la cola BFS que la garantiza únicamente cuando existe un único camino viable o cuando el camino de menor número de pasos coincide con el de menor coste y por último el DFS que es incapaz de otorgar la solución óptima a no ser que el escenario haga que todos den la misma solución y sea además la óptima.

En cuanto a los costes de movimientos, el mejor del que disponemos es el A* Ponderado sin duda alguna, es el único algoritmo que tiene en cuenta la penalización de costes de movimiento hacia Norte y Oeste y toma decisiones en consecuencia.

A pesar de esto, los algoritmos BFS y A* dan un comportamiento muy eficiente con las restricciones que cada uno posee.

5. Anexo

Una posible mejora sería implementar el algoritmo de Búsqueda de Coste Uniforme (UCS), la cual funciona igual que BFS pero logra solventar el problema de optimalidad en grafos de costes asimétricos gracias a que usa colas de prioridad ordenadas por coste acumulado $g(n)$. Su implementación sería como implementar un A* con $h(n)=0$ por lo que sería trivial y sería la solución a nuestro problema con BFS.

Otra posible mejora sería sustituir la estructura de cerrados en los 4 algoritmos por un HashSet, con el fin de reducir el coste de operaciones como “contains(s)” de $O(n)$ a $O(1)$, lo que mejoraría el rendimiento notablemente en mapas de gran tamaño.